Checksum

Error Detection Technique

CHECKSUM

The error detection method used by most TCP/IP protocols is called the checksum.

The checksum protects against the corruption that may occur during the transmission of a packet.

It is redundant information added to the packet.

The **checksum is calculated at the sender and the** value obtained is sent with the packet.

The receiver repeats the same calculation on the whole packet including the checksum. If the result is satisfactory the packet is accepted; otherwise, it is rejected.

CHECKSUM

The checksum is used in the Internet by several protocols

One's Complement
Internet Checksum

Example

Suppose our data is a list of five 4-bit numbers that we want to send to a destination.

In addition to sending these numbers, we send the sum of the numbers.

For example,
if the set of numbers is (7, 11, 12, 0, 6),
we send (7, 11, 12, 0, 6, 36),

where 36 is the sum of the original numbers.

- The receiver adds the five numbers and compares the result with the sum.
- If the two are the same, the receiver assumes no error, accepts the five numbers, and discards the sum.
- Otherwise, there is an error somewhere and the data are not accepted.

Example

We can make the job of the receiver easier if we send the negative (complement) of the sum, called the checksum.

In this case, we send (7, 11, 12, 0, 6, −36).

- ✓ *The receiver can add all the numbers received (including the checksum).*
- ✓ *If the result is 0, it assumes no error; otherwise, there is an error.*

Example

How can we represent the number 36 in one's complement arithmetic using only four bits?

Solution

The number 36 in binary is 100100 (it needs six bits).

We can wrap the extra leftmost bit and add it to the four rightmost bits.

We have (0100 + 10) = 0110 or 6.

Example

How can we represent the number −6 in one's complement arithmetic using only four bits?

Solution

In one's complement arithmetic, the negative or complement of a number is found by inverting all bits.

Positive 6 is 0110; ➔ negative 6 is 1001, this is 9.
In other words, the complement of 6 is 9.

Another way to find the complement of a number in one's complement arithmetic is to subtract the number from $2^n - 1$ (16 − 1 =15 in this case).

Example 10.22

Let us redo previous example using one's complement arithmetic.

if the set of numbers is (7, 11, 12, 0, 6)
we send (7, 11, 12, 0, 6, 36),

The result is 36. However, 36 cannot be expressed in 4 bits.

The extra two bits are wrapped and added with the sum to create the wrapped sum value 6.

The sum is then complemented, resulting in the checksum value 9 (15 − 6 = 9).

The sender now sends six data items to the receiver including the checksum 9. i.e. *(7, 11, 12, 0, 6, 9),*

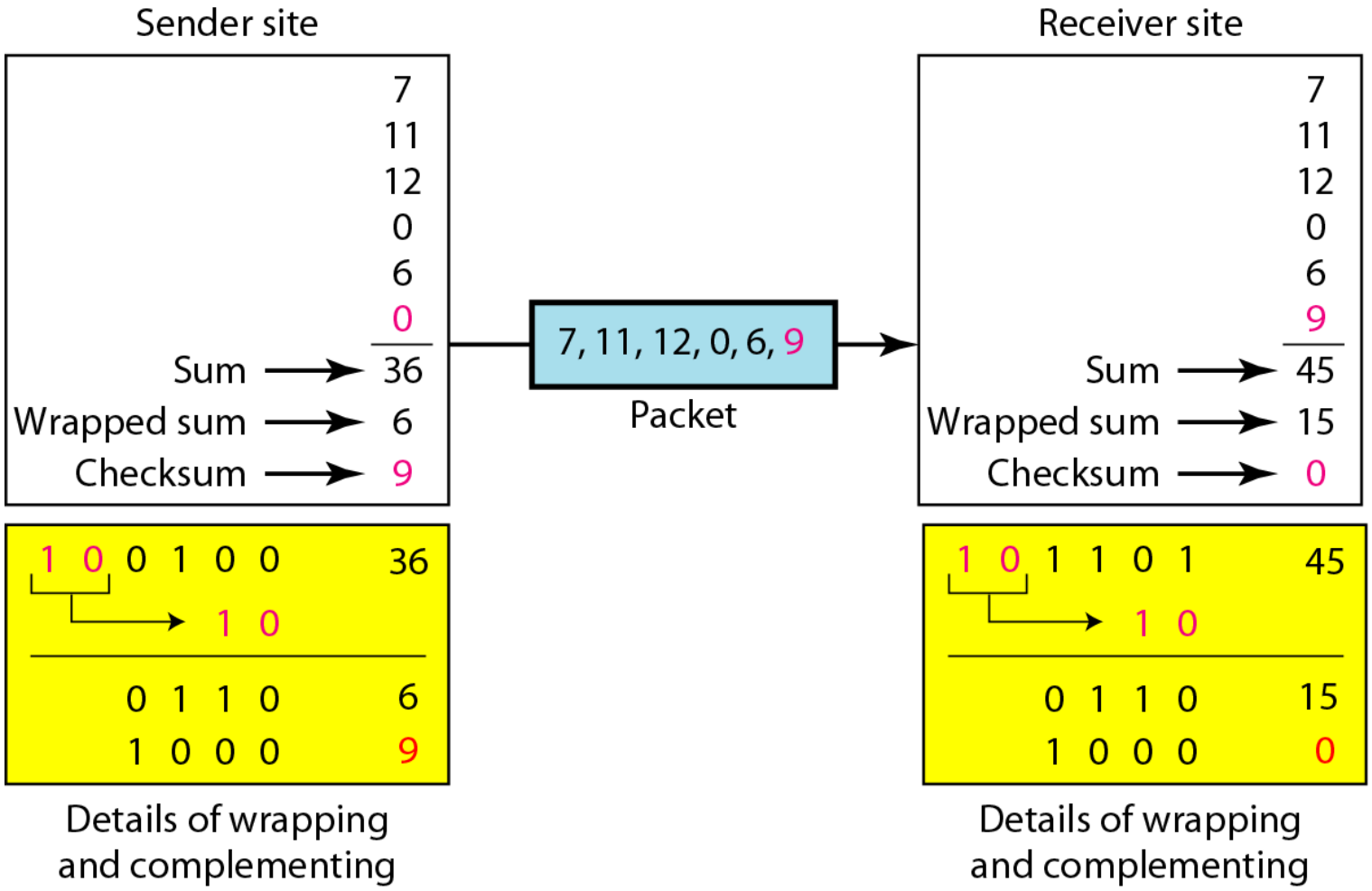
Example 10.22 (continued)

The receiver follows the same procedure as the sender.

It adds all data items (including the checksum); *(7, 11, 12, 0, 6, 9)*,
➔ the result is 45 ➔ (101101)

The sum is wrapped and becomes 15. [1101 + 10 = 1111]

- **The wrapped sum is complemented and becomes 0, this means that the data is not corrupted.**
- **The receiver drops the checksum and keeps the other data items.**
- **If the checksum is not zero, the entire packet is dropped.**


Sender site
7
11
12
0
6
0
Sum
36
Wrapped sum
6
Checksum
9
7, 11, 12, 0, 6, 9
Packet
Receiver site
7
11
12
0
6
9
Sum
45
Wrapped sum
15
Checksum
0
1 0 0 1 0 0 36
1 0
0 1 1 0 6
1 0 0 0 9
Details of wrapping and complementing
1 0 1 1 0 1 45
1 0
0 1 1 0 15
1 0 0 0 0
Details of wrapping and complementing

IP checksum calculation

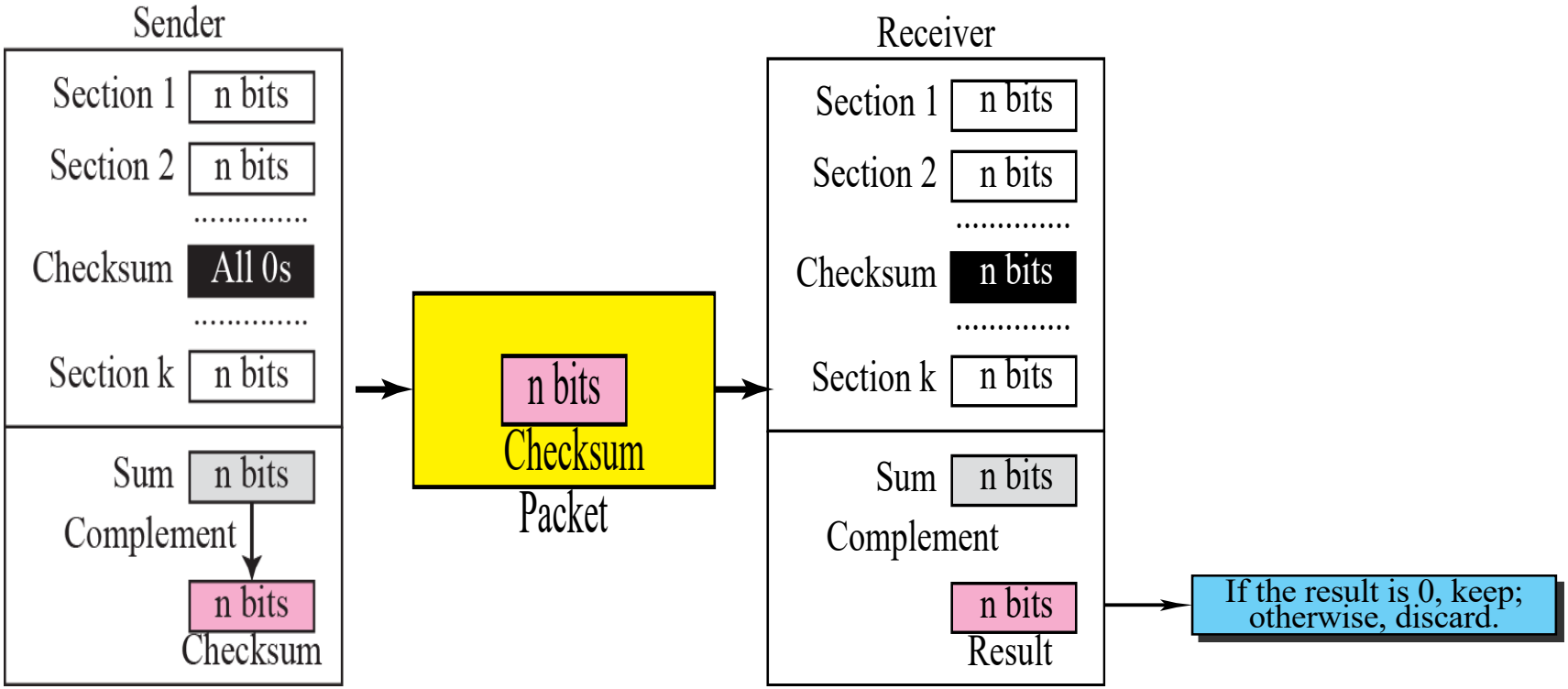
Sender site:

1. **The message is divided into 16-bit words.**
2. **The value of the checksum word is set to 0.**
3. **All words including the checksum are added using one's complement addition.**
4. **The sum is complemented and becomes the checksum.**
5. **The checksum is sent with the data.**

Receiver site:

1. **The message (including checksum) is divided into 16-bit words.**
2. **All words are added using one's complement addition.**
3. **The sum is complemented and becomes the new checksum.**
4. **If the value of checksum is 0, the message is accepted;**
5. **otherwise, it is rejected.**

Figure *Checksum concept used at IP*



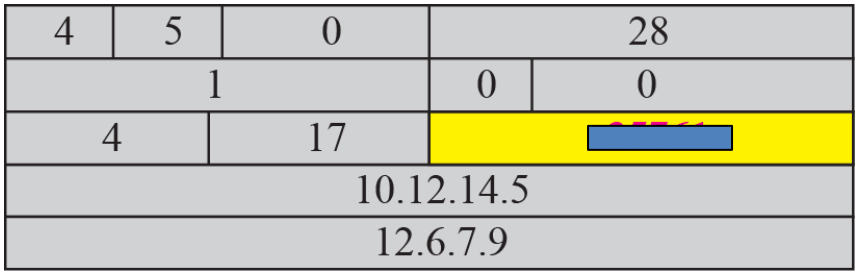
Checksum in IP covers only the header, not the data.

Example of checksum calculation at the sender

checksum calculation at the sender site for an IP header without options.

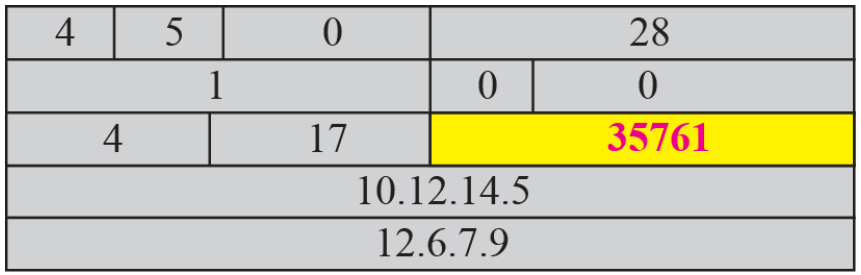
4, 5, and 0	→	01000101 00000000
28	→	00000000 00011100
1	→	00000000 00000001
0 and 0	→	00000000 00000000
4 and 17	→	00000100 00010001
0	→	00000000 00000000
10.12	→	00001010 00001100
14.5	→	00001110 00000101
12.6	→	00001100 00000110
7.9	→	00000111 00001001
Sum	→	01110100 01001110
Checksum	→	10001011 10110001

- ✓ The header is divided into 16-bit sections.
- ✓ All the sections are added
- ✓ the sum is complemented.
- ✓ The result is inserted in the checksum field



checking of checksum calculation at the receiver site (or intermediate router)

4, 5, and 0	→	01000101 00000000
28	→	00000000 00011100
1	→	00000000 00000001
0 and 0	→	00000000 00000000
4 and 17	→	00000100 00010001
Checksum	→	10001011 10110001
10.12	→	00001010 00001100
14.5	→	00001110 00000101
12.6	→	00001100 00000110
7.9	→	00000111 00001001
Sum	→	1111 1111 1111 1111
Checksum	→	0000 0000 0000 0000



- The header is divided into 16-bit sections.
- All the sections are added and the sum is complemented.
- Since the result is 16 0s, the packet is accepted.